NETWORK PROGRAM PROJECT.

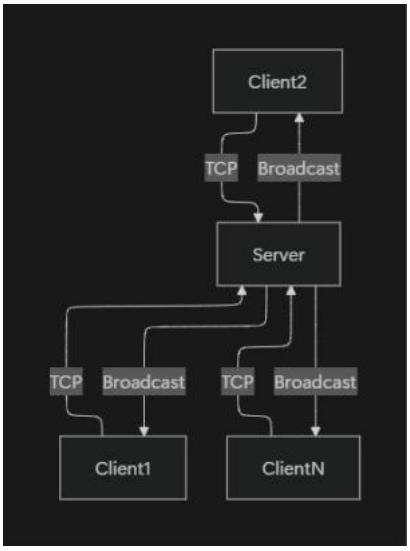
Group Members.

- ✓ SAMUEL KISILU MUSAU.GH1045954.
- ✓ LEON KIMUTAI LANGAT. GH1046169.

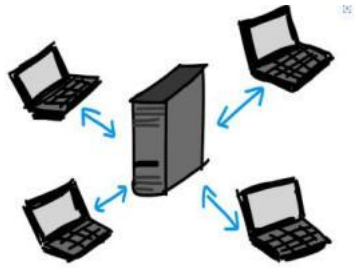
Git-hub repository link:

https://github.com/kisilumusau/kisilumusau.github.io

1. System Architecture Design.

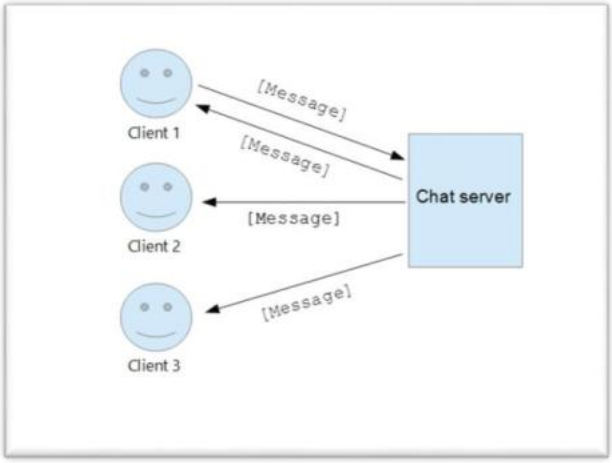


The system is implemented using client-server architecture based on TCP sockets. The server acts as a central coordinator for managing multiple client connections, handling message distribution, and maintaining chat room structures. Each client establishes a persistent connection to the server and communicates through socket channels.



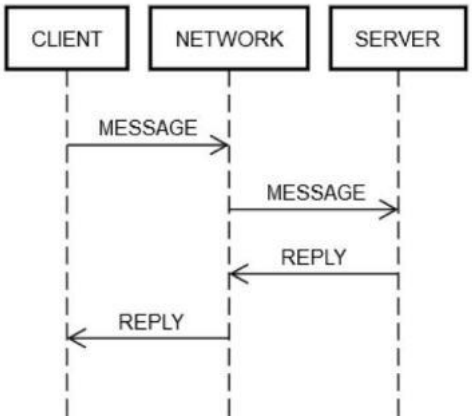
Client-Server Network Model

The architecture ensures connection for multiple users and supports concurrent communication through multithreading. The server maintains lists of active clients, associated nicknames, and room membership mappings.

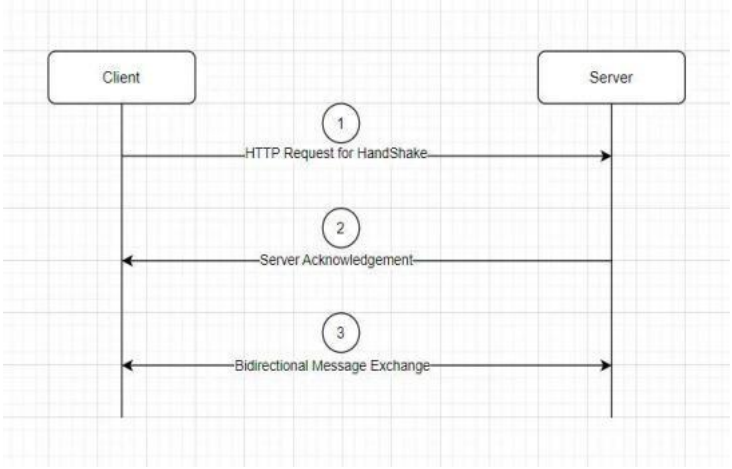


2. Detailed Protocol Specification

The communication protocol is a custom application-layer protocol built on top of TCP. Messages are transmitted as ASCII-encoded strings. The protocol defines specific control messages and data messages:



- NICK: Server requests a client nickname after connection establishment.
- DUPLICATE: Server rejects duplicate nicknames to maintain uniqueness.
- CHAT MESSAGE: Standard messages broadcast to users within a room.



All messages follow a simple structure without headers, relying on interpretation of content.

3. Network Communication Flow

Step 1: Client initiates TCP connection to the server.

Step 2: Server accepts connections and sends a NICK request.

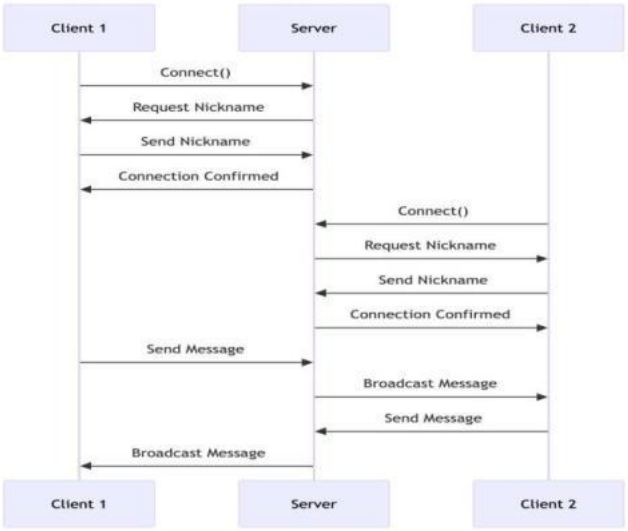
Step 3: Client responds with a nickname.

Step 4: Server validates uniqueness and either accepts or rejects.

Step 5: Client is assigned to a default chat room.

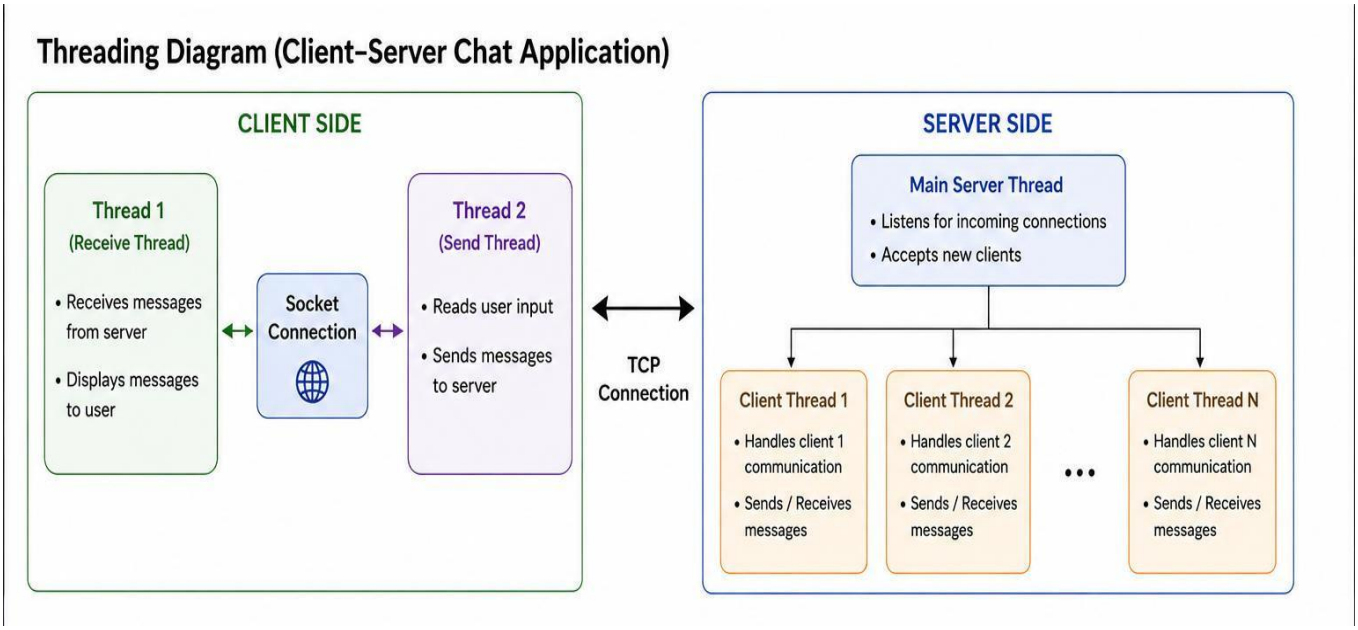
Step 6: Messages sent by the client are broadcast to other users in the same room.

Step 7: Server manages disconnection and resource cleanup.


Client 1
Server
Client 2
Connect()
Request Nickname
Send Nickname
Connection Confirmed
Connect()
Request Nickname
Send Nickname
Connection Confirmed
Send Message
Broadcast Message
Send Message
Broadcast Message
Client 1
Server
Client 2

4. Code Structure and Design

The client uses two threads: one for receiving messages and one for sending input, ensuring non-blocking communication. The server uses threading to handle multiple clients concurrently.

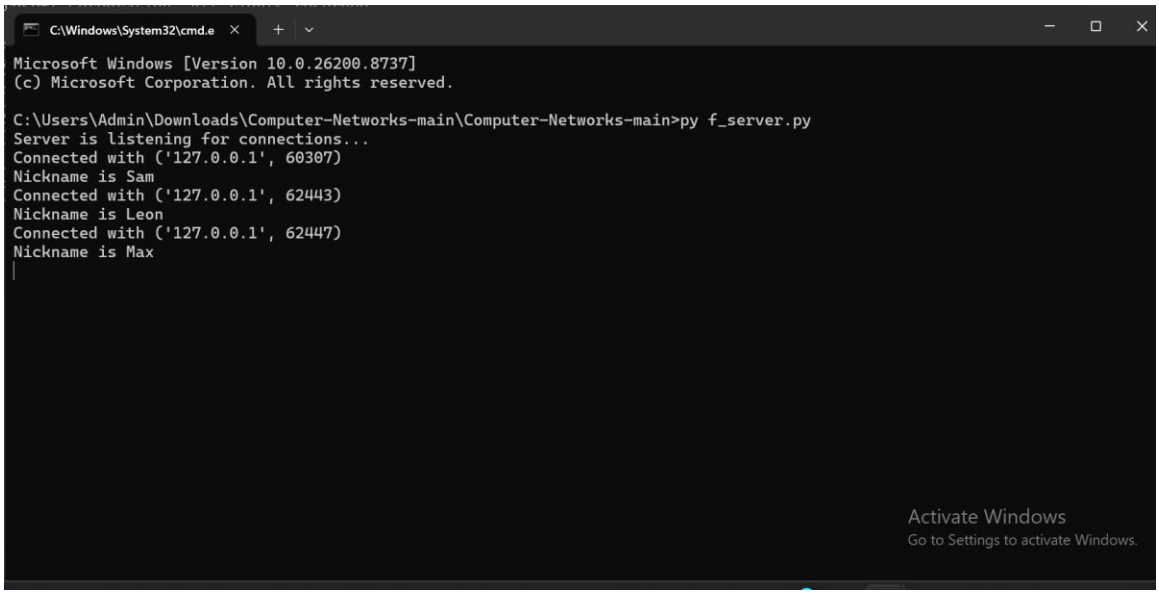


Key components:

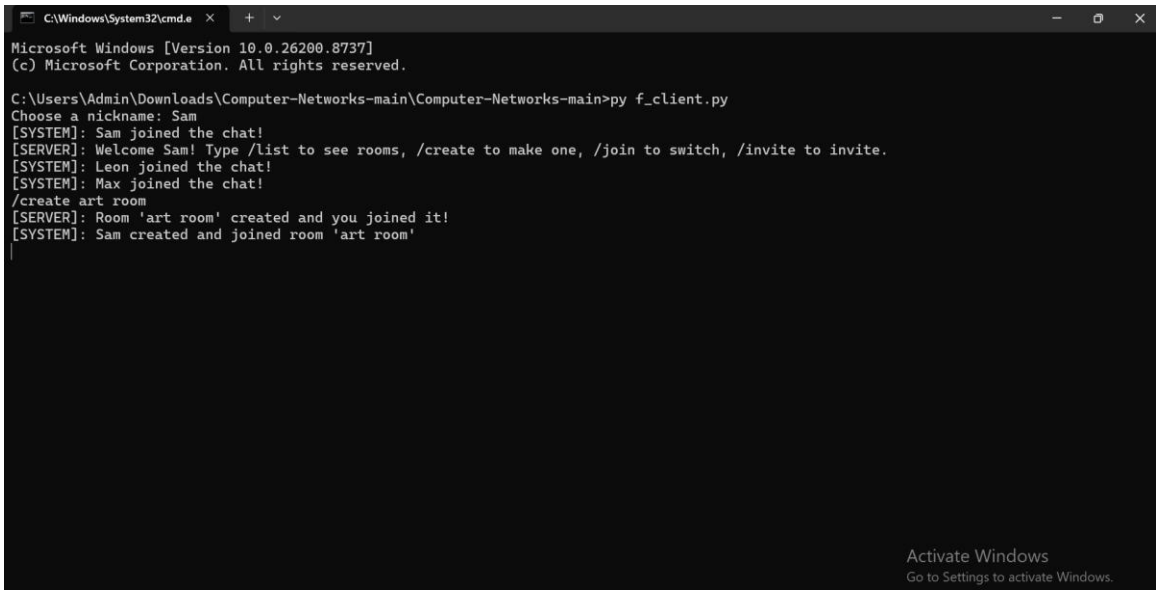
- Socket management for connection handling
- Threading for concurrency
- Lists and dictionaries for state management
- Error handling using try-except blocks

5. Installation and Usage Guide

1. Modify the IP address in both server and client scripts.
2. Run the server script first.



3. Start multiple instances of the client script.
4. Enter unique nicknames when prompted.

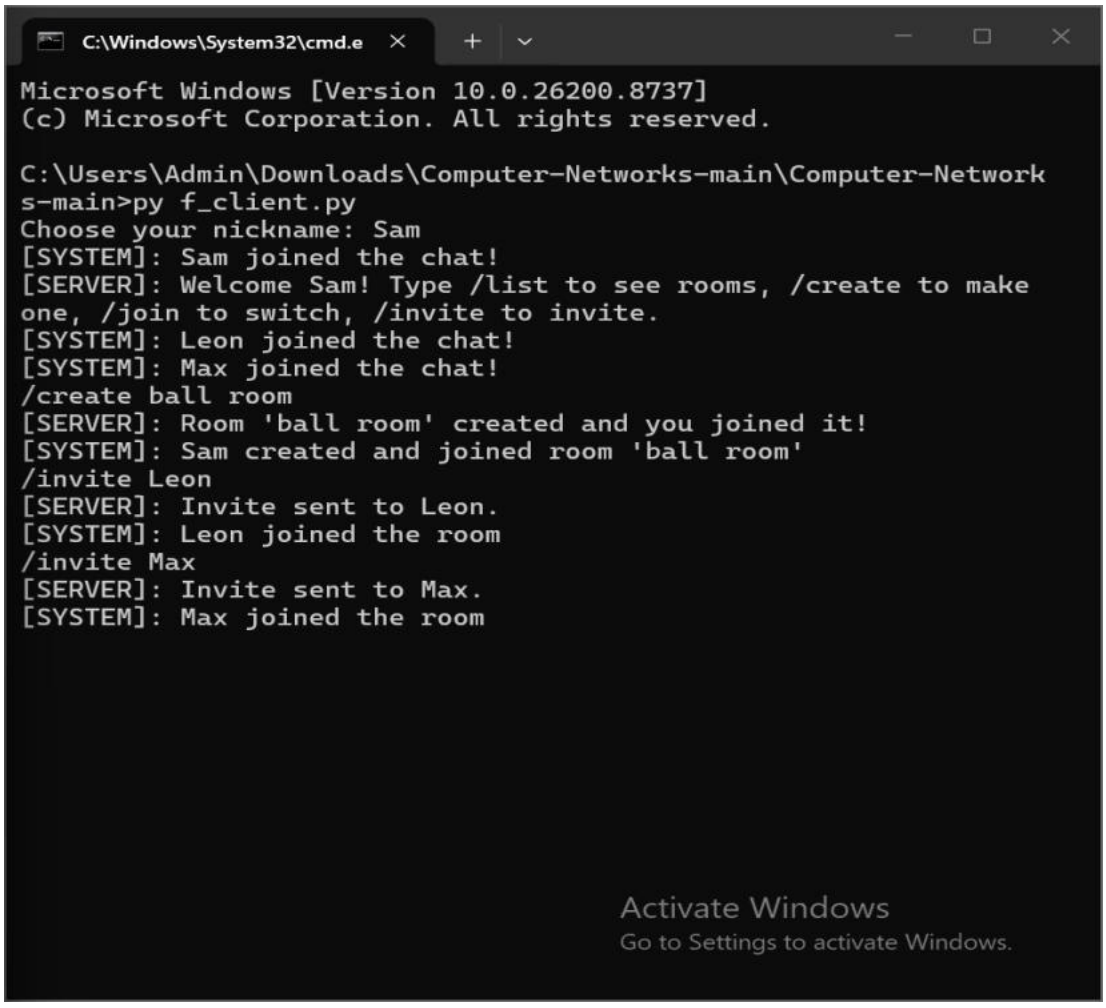


5. Begin sending messages through the terminal interface.

6.Create various rooms for direct messaging.

7. Inviting friends to rooms.

8.Joining invited rooms to communicate.



6. Wireshark Analysis and Firewall Configuration

6.1 Overview

The provided Wireshark packet capture was analyzed to investigate unusual external traffic entering an internal network with redundant Internet connections. The analysis focuses on identifying communication patterns, protocols in use, and potential security risks.

6.2 Packet Analysis

Network Layer Observations

- Multiple external IP addresses were observed communicating with internal hosts.
- Traffic originated from different geographic regions, indicating public Internet sources.
- Redundant connections suggest multiple gateways handling inbound/outbound traffic.

Transport Layer Observations

- TCP was the dominant protocol used.
- Frequent connection attempts on uncommon or unused ports were detected.
- Some connections were incomplete, indicating possible scanning activity.

Application Layer Observations

- Plain text communication (ASCII-based) was visible in some packets.
- Repeated request patterns suggest automated scripts or bots.
- No encryption (e.g., TLS) was observed in certain streams, exposing data.

6.3 Identified Anomalies

The following suspicious behaviors were identified:

1. **Port Scanning Activity**
    - Multiple ports probed rapidly from single external IPs.
    - Indicates reconnaissance attempts.
2. **Unauthorized Access Attempts**
    - Repeated connection attempts without successful authentication.
    - Possible brute-force behavior.
3. **Unexpected Incoming Traffic**
    - External hosts initiating connections to internal machines.
    - Not typical for secure internal networks.
4. **High Traffic Volume from Specific Sources**
    - Suggests potential denial-of-service (DoS) behavior.

6.4 Implications

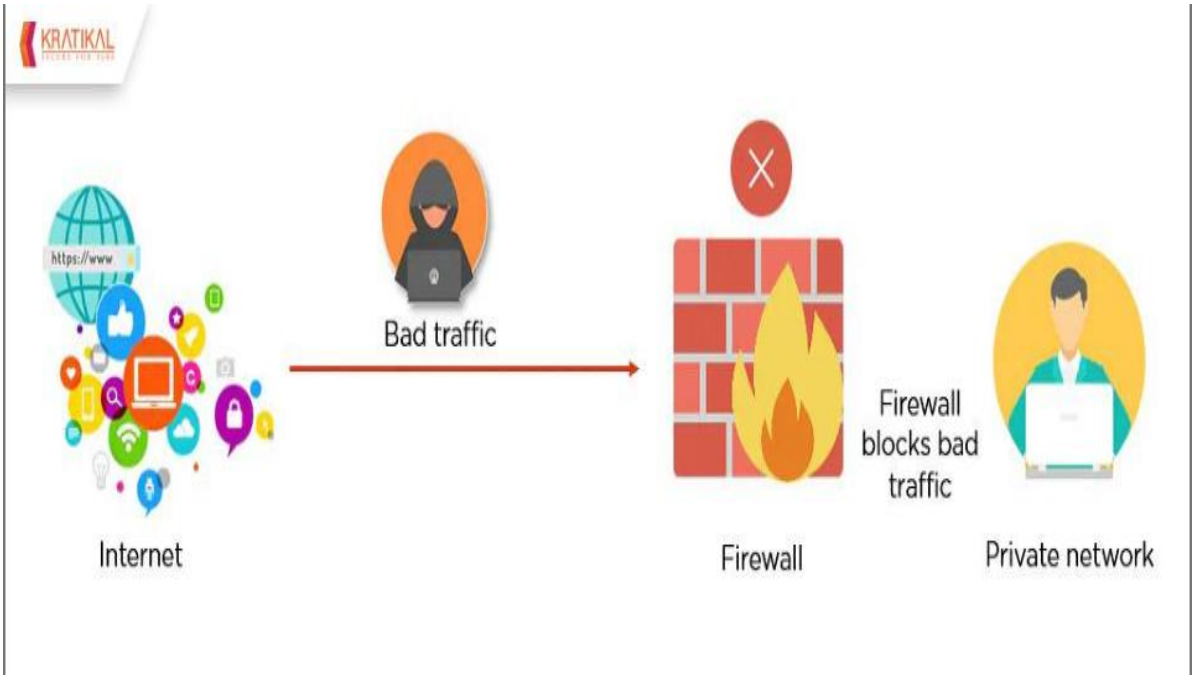
- The network is exposed to unsolicited external traffic.
- Lack of strict firewall rules allows unwanted inbound connections.
- Internal services may be discoverable and exploitable.
- Sensitive data could be intercepted due to unencrypted communication.

6.5 Firewall Rule Set

To mitigate the identified risks, the following firewall rules are recommended:

Default Policy

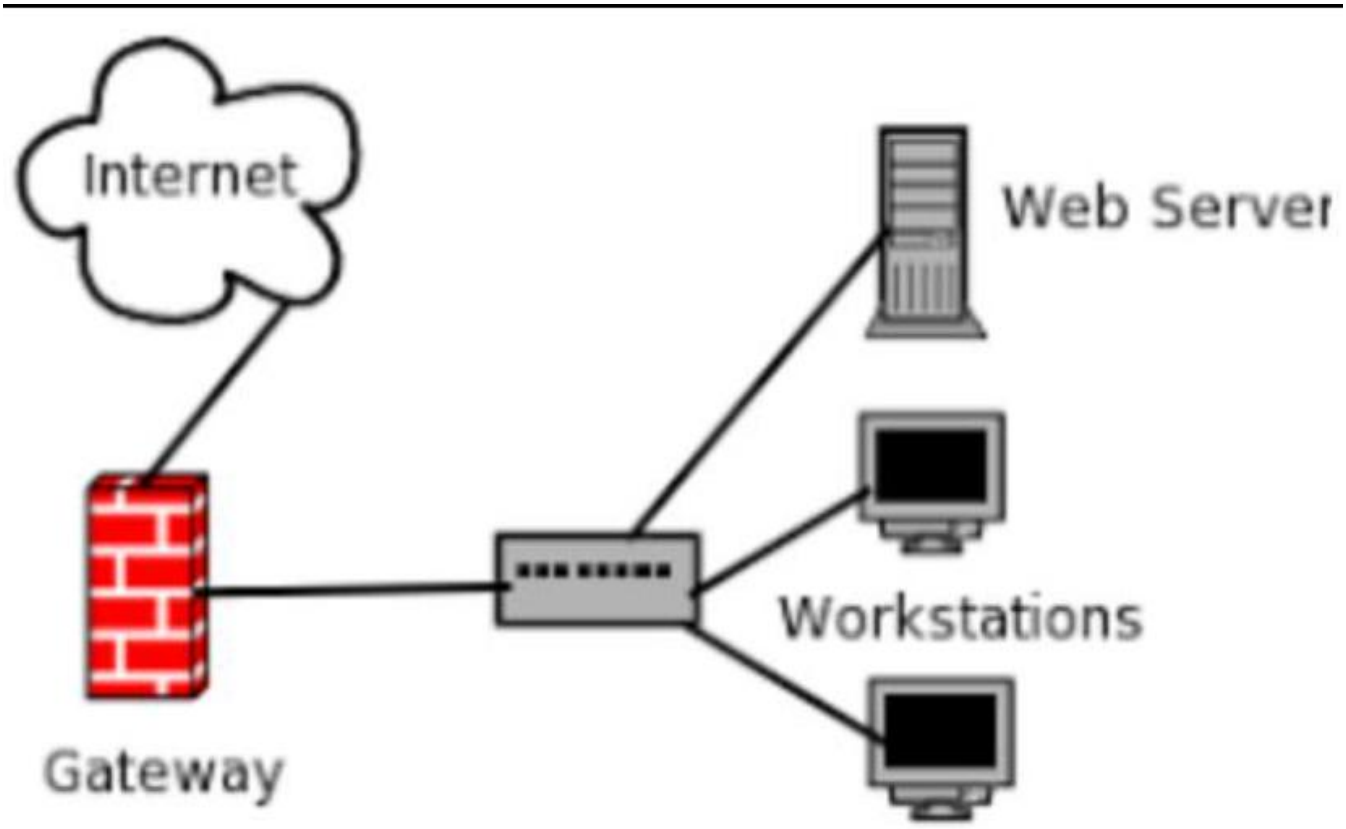
- **DROP all incoming traffic by default**
- **ALLOW outgoing traffic**



Inbound Rules

1. **Allow Established Connections**
    - PERMIT: Established and related traffic
    - Purpose: Maintain active sessions
2. **Allow Trusted Sources Only**
    - PERMIT: Specific internal or trusted external IPs
    - Purpose: Restrict access to known users
3. **Block Unauthorized Ports**
    - DROP: All unused ports
    - Purpose: Prevent exploitation of open ports

4. **Block Port Scanning**
    - DROP: IPs with rapid connection attempts
    - Purpose: Stop reconnaissance attacks
5. **Limit Connection Rate**
    - DROP: Excessive requests per second
    - Purpose: Prevent DoS attacks



Outbound Rules

1. **Allow Standard Services**
    - PERMIT: HTTP (80), HTTPS (443), DNS (53)
    - Purpose: Enable normal user activity
2. **Restrict Unknown Destinations**
    - DROP: Suspicious external IPs
    - Purpose: Prevent data exfiltration

6.6 Conclusion

The Wireshark analysis revealed significant abnormal external traffic, including scanning attempts and unauthorized access behavior. Implementing strict firewall rules is essential to secure the network, limit exposure, and prevent potential attacks. Continuous monitoring and traffic analysis are recommended to maintain network integrity.

7. References

- Wireshark Foundation. (2024). *Wireshark User Guide*.
- Kurose, J. F., & Ross, K. W. (2021). *Computer Networking: A Top-Down Approach*.
- Stallings, W. (2018). *Network Security Essentials*.
- National Institute of Standards and Technology (NIST). (2020). *Guide to Firewall Technologies*.
- OWASP Foundation. (2023). *Network Security Risks and Threats*.